Module 1: Introduction to Python

Concept

Python is a high-level, interpreted, general-purpose programming language known for simple and readable syntax.

Why Python for AI?

- •Easy syntax
- •Large AI ecosystem
- •Many libraries
- •Fast development
- •Used for AI, ML, Data Science, Automation, Web Development, and APIs

First Program

```
print("Hello, AI World!")
```

Example 2

```
print("Welcome to the AI Bootcamp") print("Learning Python for AI")
```

Example 3

name = "Rahul" print("Hello", name)

Coding Questions

1.Print your name, college, and branch.
2.Print "Welcome to Generative AI Bootcamp" five times.
3.Print your dream company.
4.Print a simple student profile.
5.Print the following pattern:

*

**

***

****

*****

Module 2: Variables

Definition

A variable is a named location used to store a value in memory.

Syntax

variable_name = value

Example 1

name = "Rahul" age = 20 print(name) print(age)

Example 2

model_name = "GPT" temperature = 0.7 print(model_name) print(temperature)

Example 3

student_name = "Aman" cgpa = 8.4 is_placed = False print(student_name) print(cgpa) print(is_placed)

Industry Connection

In an AI application, variables may store:

model = "GPT" prompt = "Explain Python" response = "Python is easy to learn"

.

Coding Questions

1.Store and display your name, age, and city.
2.Store marks in five subjects and print them.
3.Store the name of an AI model and display it.
4.Swap two numbers using a temporary variable.
5.Swap two numbers without using a temporary variable

Module 3: Data Types

Definition

A data type specifies the type of value stored in a variable.

Common Python Data Types

Data Type	Example
int	25
float	8.5
str	"Python"
bool	True
list	["Python", "AI"]
tuple	("BCA", "MCA")
set	{"AI", "ML"}
dict	{"name": "Rahul"}

Example 1

```
age = 20
print(type(age))
```

Example 2

```
cgpa = 8.5
print(type(cgpa))
```

Example 3

```
language = "Python"
print(type(language))
```

Example 4

```
is_placed = True
print(type(is_placed))
```

Coding Questions

1.Create one variable of every basic data type.

2.Print the data type of five different values.

3.Store student information using suitable data types.

4.Convert an integer into a float.

5.Convert a numeric string into an integer.

Module 4: Input and Output

Definition

- input() accepts data from the user.
- print() displays output.

Example 1

```
name = input("Enter your name: ")
print("Welcome", name)
```

Example 2

```
age = int(input("Enter your age: "))
print("Your age is:", age)
```

Example 3

```
number1 = int(input("Enter first number: "))
number2 = int(input("Enter second number: "))
print("Sum:", number1 + number2)
```

Important Point

input() returns data as a **string** by default.

Coding Questions

1.Take a student's name and display a welcome message.
2.Take two numbers and calculate their sum.
3.Take the length and width of a rectangle and calculate its area.
4.Take marks in five subjects and calculate the percentage.
5.Take a user's name and favourite AI tool and display both.

Module 5: Type Conversion

Definition

Type conversion changes a value from one data type to another.

Example 1

```
number = "25"
converted_number = int(number)
print(converted_number)
print(type(converted_number))
```

Example 2

```
number = 20
decimal_number = float(number)
print(decimal_number)
```

Example 3

```
age = 20
message = "My age is " + str(age)
print(message)
```

Coding Questions

1.Convert a string into an integer.

2.Convert an integer into a float.

3.Convert a float into an integer.

4.Take two numbers as input and calculate their sum.

5.Convert a number into a string and print a message.

Module 6: Operators

Types of Operators

1. Arithmetic Operators

```
a = 20
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a ** b)
print(a // b)
```

2. Comparison Operators

```
a = 10
b = 20
print(a == b)
print(a != b)
print(a > b)
print(a < b)
```

3. Logical Operators

```
age = 20
has_id = True
print(age >= 18 and has_id)
```

Coding Questions

1.Build a simple calculator.
2.Calculate the area and perimeter of a rectangle.
3.Calculate the percentage of five subjects.
4.Check whether two numbers are equal.
5.Check whether a student is eligible using age and marks.

Module 7: Conditional Statements

Definition

Conditional statements allow a program to make decisions.

if

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

if-else

```
number = 10
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

if-elif-else

```
marks = 85
if marks >= 90:
    print("Grade A+")
elif marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Needs Improvement")
```

AI Connection

AI applications also make decisions based on conditions.

```
confidence = 0.92
if confidence >= 0.80:
    print("Prediction Accepted")
else:
    print("Human Review Required")
```

Coding Questions

1.Check whether a number is even or odd.
2.Check whether a person is eligible to vote.
3.Find the largest of two numbers.
4.Find the largest of three numbers.
5.Create a student grade calculator.

Module 8: Loops

Loops repeat a block of code.

for Loop

```
for i in range(1, 6):
    print(i)
```

Output

```
1
2
3
4
5
```

while Loop

```
number = 1
while number <= 5:
    print(number)
    number += 1
```

Loop With a List

```
ai_tools = ["ChatGPT", "Gemini", "Claude"]
for tool in ai_tools:
    print(tool)
```

Coding Questions

1.Print numbers from 1 to 50.
2.Print even numbers from 1 to 50.
3.Print odd numbers from 1 to 50.
4.Print the multiplication table of a number.
5.Calculate the sum of numbers from 1 to n.

Module 9: break, continue, and pass

break

Stops the loop immediately.

```
for i in range(1, 11):
    if i == 6:
        break
    print(i)
```

continue

Skips the current iteration.

```
for i in range(1, 11):
    if i == 5:
        continue
    print(i)
```

pass

Creates an empty code block temporarily.

```
for i in range(5):
    pass
```

Coding Questions

1.Stop a loop when the number becomes 10.
2.Print numbers from 1 to 20 but skip multiples of 3.
3.Search for a number in a list and stop when it is found.
4.Print numbers until the user enters 0.
5.Create a simple password-attempt program.

Module 10: Strings

A string is a sequence of characters.

Example

```
message = "Learning Python for AI"
print(message)
```

String Indexing

```
language = "Python"
print(language[0])
print(language[1])
```

String Methods

```
text = "python for ai"
print(text.upper())
print(text.lower())
print(text.title())
```

Coding Questions

1.Find the length of a string.

2.Count vowels in a string.

3.Reverse a string.

4.Check whether a string is a palindrome.

5.Count the number of words in a sentence.

Module 11: Lists

A list is an ordered and mutable collection of values.

Example

```
ai_tools = ["ChatGPT", "Gemini", "Claude"]
print(ai_tools)
```

Accessing Elements

```
print(ai_tools[0])
print(ai_tools[1])
```

Adding an Item

```
ai_tools.append("Perplexity")
print(ai_tools)
```

Removing an Item

```
ai_tools.remove("Gemini")
print(ai_tools)
```

Coding Questions

1.Store five programming languages in a list.
2.Find the largest element in a list.
3.Find the smallest element in a list.
4.Calculate the sum of all elements.
5.Remove duplicate values from a list.

Module 12: Tuples

A tuple is an ordered and immutable collection.

Example

```
student = ("Rahul", 20, "BCA")
print(student)
```

Accessing Values

```
print(student[0])
```

Coding Questions

1. Create a tuple containing five programming languages.
2. Find the length of a tuple.
3. Access the first and last element.
4. Count the occurrence of an element.
5. Convert a tuple into a list.

Module 13: Sets

A set stores unique values.

Example

```
skills = {"Python", "AI", "Python", "ML"}
print(skills)
```

The duplicate "Python" is removed.

Operations

```
skills.add("NLP")
skills.remove("AI")
print(skills)
```

Coding Questions

1.Remove duplicate values from a list using a set.
2.Find common elements between two sets.
3.Find all unique elements from two sets.
4.Add and remove elements from a set.
5.Count unique programming languages.

Module 14: Dictionaries

A dictionary stores data in **key-value pairs**.

Example

```
student = {
    "name": "Rahul",
    "branch": "BCA",
    "cgpa": 8.5
}
print(student)
```

Accessing Values

```
print(student["name"])
```

Adding Data

```
student["college"] = "BIM"
print(student)
```

AI Connection

API responses are commonly structured like JSON, which maps naturally to Python dictionaries.

```
response = {
    "model": "GPT",
    "answer": "Python is useful for AI"
}
print(response["answer"])
```

Coding Questions

1.Create a dictionary containing student information.
2.Add a new key-value pair.
3.Update a student's CGPA.
4.Display all keys and values.
5.Create a dictionary of five subjects and marks.

Module 15: Functions

A function is a reusable block of code designed to perform a specific task.

Function Without Parameters

```
def welcome():
    print("Welcome to AI Bootcamp")
welcome()
```

Function With Parameters

```
def greet(name):
    print("Hello", name)
greet("Rahul")
```

Function With Return

```
def add(a, b):
    return a + b
result = add(10, 20)
print(result)
```

Coding Questions

1.Create a function to add two numbers.
2.Create a function to find the square of a number.
3.Create a function to check even or odd.
4.Create a function to find the largest of two numbers.
5.Create a function to calculate the factorial.